FILE INTEGRITY MONITORING / FIELD GUIDE

SentinelFIM Project Guide

Architecture, SHA-256, alerts, operation, testing, and practical limits

Educational defensive cybersecurity application

A source-verified guide to how this Python and PyQt6 desktop project builds file fingerprints, compares them with a trusted baseline, and presents changes for human review.

SEE WHAT CHANGED • REVIEW THE EVIDENCE • UNDERSTAND THE LIMITS

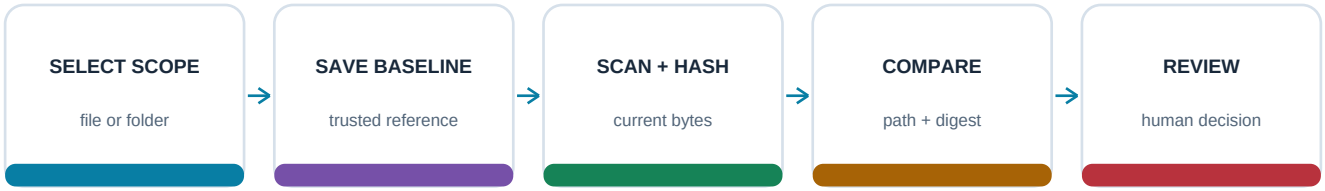
This guide explains the implementation in the repository. It is not a certification claim or a promise that a file is safe.

1. What this project is

SentinelFIM is a local desktop File Integrity Monitoring (FIM) application. A user selects files or a directory, saves a trusted snapshot called a baseline, then compares later scans against that reference. It reports new, changed, missing, or matching-fingerprint moved files.

Why it is useful

Unexpected changes to configuration, application binaries, release files, or records can signal a failed deployment, accidental edit, unauthorized change, or other event worth checking. FIM helps surface the difference. It does not determine motive, identify the person or process responsible, or prevent the change.



A baseline is an expected reference, not proof that its contents are safe.

Audience	Example value	Important boundary
Students and developers	Learn hashing, baseline comparison, event handling, and evidence review.	A learning application, not an enterprise fleet platform.
IT administrators	Review unexpected changes to a small, selected set of files.	Requires the app to remain running and scope to be configured.
Security analysts	Use alerts as a starting point for triage and corroboration.	An integrity difference alone does not prove malicious activity.

Industry context

The FIM control category is relevant in financial services, healthcare, government, cloud operations, and software delivery. This guide uses these as illustrative contexts only; it does not claim SentinelFIM is deployed, certified, or compliant in those industries.

2. Architecture and data flow

The application uses a PyQt6 desktop interface and a background monitor worker. The worker walks the selected scope, reads file bytes in bounded chunks, computes digests, compares the current inventory to the active baseline, then returns scan results to the UI.

Watchdog notifications can wake an early scan; periodic scans still recompute hashes and compare results.

Implementation facts verified in app.py

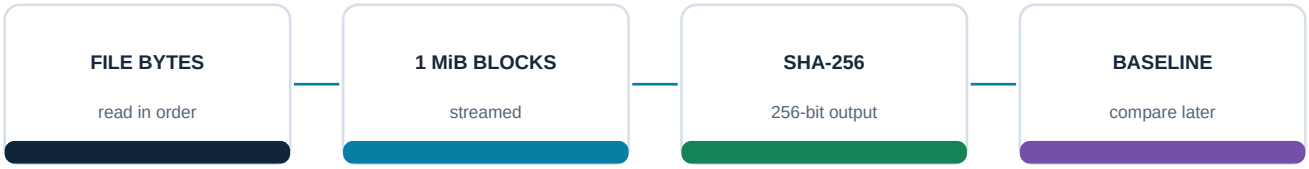
- Recursive scanning uses normalized relative paths, so same-named files in different subfolders remain separate records.
- Regular files are read in 1 MiB chunks. SHA-256 and SHA-512 are calculated side by side in the same read pass.
- The scanner enforces a 5,000-file cap and a configurable per-file size ceiling (1 GiB default). It also bounds cached text used for forensic diffs.
- Watchdog events wake the worker; periodic scans and hash comparison determine the integrity result. The application does not block writes at the operating-system level.
- Access/enumeration issues are reported and known files under uncertain paths are retained for the session rather than declared deleted. Symlinks are not followed and are reported.

Baseline authentication

The JSON baseline database is authenticated with HMAC-SHA-256. Its signing key is derived from the user's password using scrypt. This detects unauthorized edits when the secret remains protected; it does not encrypt the JSON data or make a same-machine baseline independent of that machine.

3. How SHA-256 works in this project

SHA-256 is a one-way cryptographic hash function specified by NIST. It accepts a message of any practical length and produces a 256-bit (32-byte) message digest. The digest is commonly written as 64 hexadecimal characters.



Same bytes -> same digest. A changed digest proves different input bytes, not why they changed.

The monitor reads the exact file bytes and updates the SHA-256 and SHA-512 hash states as each chunk arrives. It stores the resulting digest with the normalized relative path in the current scan and baseline records. Later it computes the digest again and compares it with the saved value.

Worked example

```
INPUT A (UTF-8): Release version: 1.0\n
SHA-256: 2a3a3a6e9034de271766ffd5bfebe99ce66aa7ae6a4ad65ce5e7586e8533e0b0

INPUT B (UTF-8): Release version: 1.1\n
SHA-256: 1adc7f6c0e7e077fa5005a3f43ea0e8abfd1c4b2903e538cb45dbcb194e9c47c
```

What the result tells you

Different digests show that the input bytes differ. A digest does not reveal which byte changed, who changed it, or whether the change was malicious. This application may show a text diff when the content was captured within its bounded text cache; binary or large files can show hash evidence without a text diff.

Changing one character changes the digest with overwhelming probability. Hashes are not reversible encryption and do not recover the file contents.

4. SHA-256 and SHA-512

SHA-256 and SHA-512 belong to the SHA-2 family defined by NIST. Both create fixed-length digests. SHA-512 produces a longer digest; it is not a risk rating or an alert severity.

Property	SHA-256	SHA-512
Digest size	256 bits (32 bytes)	512 bits (64 bytes)
Hex display	64 characters	128 characters
Project behavior	Computed from the file bytes during scanning.	Computed alongside SHA-256 during the same scan pass.
Comparison meaning	Different value means current bytes differ from baseline.	Different value means current bytes differ from baseline.
Practical reading	Shorter output; easier to display and store.	Longer output; larger digest space, with more output to store/display.

Why calculate both?

Side-by-side hashes are useful for demonstrating digest lengths and confirming the scanner maintains two independent SHA-2 fingerprints. For ordinary integrity comparison, SHA-256 is a strong general-purpose choice. Calculating both does not double protection, and an attacker who can alter the file and trusted baseline can undermine local comparison.

Hash vs. HMAC

A file hash is an unkeyed fingerprint of file bytes. HMAC is a keyed message authentication code. SentinelFIM uses HMAC-SHA-256 to authenticate its baseline database, with a key derived using scrypt. These solve related but distinct problems: file hashes support comparison; HMAC checks whether stored baseline data has been altered without the key.

Neither hash calculates whether a file is safe. A match says the bytes match the chosen baseline; baseline quality depends on how and when it was created and protected.

5. Operation and quick start

Full install notes and troubleshooting belong in the repository README. The steps below show a short, controlled local verification workflow.

Step	Action	Expected result
1 · Launch	On Windows, run `run.bat` from the project folder, or `.\.venv\Scripts\python.exe app.py` after dependencies are installed.	The desktop application opens.
2 · Select scope	In the Dashboard, paste a path or choose a file/folder. Use a temporary test folder for the exercise.	The target path is shown; folder mode can include nested files.
3 · Start	Click Start monitoring and allow the first successful scan to complete.	A first baseline may be saved automatically if none exists for that target.
4 · Change a test file	Create or edit a text file inside the monitored test folder.	The next scan classifies the path against the baseline.
5 · Review	Open Alerts for Details or Investigate; use Inventory to locate the relative path and status.	Review recorded time, sizes, hashes, and scan diagnostics; decide whether the change was expected.

Environment note: the repository's Windows launcher is `run.bat`; this is the environment used for verification. A `run.command` launcher is also present for macOS/Linux, but those platforms were not validated in this test run.

Common event labels

Label	Meaning
CREATED	Current file exists but its relative path was absent from the selected baseline.
TAMPERED	File exists in both records, but a configured digest differs.
DELETED	Baseline file is absent from a successful scan. Access errors should not be treated as deletion.
RENAMED	A removed path and new path have matching SHA-256, SHA-512, and size.
SCAN ERROR	The scan could not verify the path reliably; investigate scan diagnostics.

Safe demo practice

Use a temporary directory and non-sensitive files. Stop monitoring when finished. The alert is a review prompt; it does not quarantine, restore, or block the changed file.

6. Application walkthrough

The implementation separates operational summary, baseline management, alert review, filesystem inventory, and forensic analysis. Existing application workflows remain the source of live data; this guide does not substitute mock screens.

Destination	What a reviewer can inspect
Dashboard	Monitoring state, target summary, trusted-baseline state, integrity assessment, latest scan, file-change counts, and activity/scan review.
Baseline Manager	Saved snapshots, target, stored records, creation time, and audit/delete actions. Verification is read-only and does not create a baseline.
Alerts	One event feed with Details for concise change evidence and Investigate for technical forensic context.
System Inventory	Nested relative paths, file sizes, SHA-256 and integrity state; filters distinguish changes and scan errors.
AI Assistant / Forensic Analysis	Optional incident-specific explanation. The selected incident metadata is sent only after an analyst requests it; file contents are not included in the Gemini prompt.

Screenshots to add before portfolio publication

No current, readable captures of the working application were available in the repository. Older user screenshots show a prior UI, and existing offscreen renders have unreadable font fallback; neither is presented here as a current product screenshot.

Capture	Evidence to show	Caption focus
01 · Dashboard	Current app after a successful scan, with real counters and status.	Scope, baseline, and scan are observable; no sample metrics.
02 · System Integrity	Current integrity assessment tied to a successful scan and chosen baseline.	A match rate is an observed result, not a security guarantee.
03 · Controlled alert	Temporary test file modified under a known baseline.	Show path, timestamp, before/after size and digest evidence.
04 · Investigation	Forensic view for that same test incident.	Separate observed facts from advisory interpretation.
05–06 · Inventory / baseline	Nested inventory and baseline history with the same test folder.	Show relative paths and selected snapshot/audit action.

7. Practical threat model

This compact model describes what this local desktop FIM can and cannot establish. Detection is not prevention, response, attribution, or proof of malicious intent.

Threat or failure scenario	What SentinelFIM can detect	Limitations	Recommended mitigation
Baseline data altered	HMAC verification can reject a modified database when the signing secret remains protected.	Same-host admin compromise or stolen password can defeat this trust assumption; data is not encrypted.	Protect the password and OS account; keep separate protected backups of trusted snapshots.
Unexpected file modification	A changed configured digest and size are recorded against the selected baseline.	No actor/process attribution; an authorized update also changes hashes; no write blocking.	Correlate timestamp/path with approved change records and endpoint logs.
Directory becomes inaccessible	Scan issues can be surfaced and known paths retained during the session.	An error can hide further changes; actual Windows ACL denial was not integration-tested.	Resolve permissions, inspect diagnostics, and do not accept uncertain scans as clean.
Monitoring stops or host is offline	The app reflects the worker's running state and subsequent scans when restarted.	No monitoring occurs while the app is closed; this is not a protected service or tamper-proof sensor.	Keep app active during use; use managed/central FIM for production coverage.
Legitimate update changes files	Changed hashes are still detected and can be reviewed.	FIM cannot infer whether a change was approved or malicious.	Coordinate baseline changes with controlled deployment and explicitly save a new baseline after review.

Scope boundary

SentinelFIM does not inspect network traffic, prevent writes, restore deleted files, identify the responsible person/process, or replace antivirus/EDR. Optional AI summaries are advisory and do not change local evidence classification.

8. Verification and test evidence

The repository uses Python's unittest framework. The following results are from the full suite run for this guide; do not substitute earlier project-documentation counts for the run recorded here.

Test case	Expected result	Actual result	Status
Recursive inventory / duplicate names	Nested files appear under distinct relative paths.	Validated by nested inventory and identical-name tests.	PASS
Create / modify / delete	Events match baseline comparison.	Controlled worker tests cover nested create, modify, and delete.	PASS
Scan access error	Issue is reported; uncertain entries are not called deleted.	Injected enumeration/read errors retain known records.	PASS
Baseline lifecycle	Queued first scans reuse one initial baseline; repeated alert/scan state is covered.	Regression test confirms first-scan deduplication; the full suite does not claim to exercise every interactive verify navigation path.	PASS
Gemini success / failure	Loading clears; valid success and sanitized failure are distinguished.	Mocked SDK tests cover success, timeout, 503 retry, fallback, and key redaction.	PASS
Long-running monitoring	Nested changes alert while original baseline remains fixed.	Real MonitorWorker + Watchdog test runs over 20 seconds.	PASS
Windows symlink integration	Create a real symlink and confirm it is not followed.	Skipped: current Windows account lacks symlink privilege (WinError 1314); deterministic rejection unit test passes.	SKIPPED
Gemini live network / Windows ACL	Exercise real API and actual denied-folder ACL.	Not run: no API key; actual ACL denial unavailable. Mock/injected error tests are not live integration evidence.	NOT RUN

Latest full suite result

34 tests run · 33 passed · 0 failed · 1 skipped · 32.720 s. Python bytecode compilation passed. The suite includes unit/regression coverage plus a real local long-monitoring worker test. Mocked API checks verify application behavior, not Gemini service availability.

Run from the project root (Windows):

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

The full command output and exact elapsed time are also retained as `tmp/pdfs/improved-guide-test-run.log` in the working project copy.

Test modules

Module	Verified coverage
test_app.py	Nested scanning, duplicate relative paths, change detection, scan errors, exclusions, baseline authentication, and alert lifecycle.
test_inventory.py	Status classification, nested rows, filters, and incomplete-scan handling.
test_forensics.py	Alert details/actions, baseline deduplication, themes, mocked Gemini success/failure/retry, diagnostics, and loading cleanup.
test_long_monitoring.py	Real MonitorWorker + Watchdog behavior over repeated nested changes and baseline stability.

9. Limitations, glossary, and references

Treat alerts as evidence to verify and contextualize. The app is designed for learning, local demonstrations, and small-scope monitoring; it has not been evaluated as an enterprise production control.

Known limitations

- A scan is bounded by file-count and size settings, exclusions, OS permissions, and available time. Inaccessible content remains unknown, not clean.
- The app must remain running to monitor. The watchdog is a scan wake-up aid; hashes and baseline comparison determine findings.
- Baseline database contents are authenticated but not encrypted. The same host is not an independent trust anchor.
- Text diffs are bounded by per-file and total cache limits; large/binary files may have only size/hash evidence.
- No live Gemini call, real Windows ACL denial, or privileged real-symlink integration was performed in the verified run described here.
- A hash mismatch does not identify an actor, establish intent, or prove compromise. AI output is advisory.

Glossary

Baseline: saved expected inventory. **Digest:** fixed-length hash output. **Relative path:** location from selected root. **HMAC:** keyed message authentication code. **Collision:** distinct inputs with the same digest; strong hashes are designed to make finding one computationally infeasible, not mathematically impossible.

Primary references

NIST, *FIPS 180-4: Secure Hash Standard*. SHA-256 and SHA-512 definitions. Official NIST publication.

NIST, *Hash Functions Project*. Approved hash families and properties. Official project page.

NIST, *SP 800-53 Rev. 5, SI-7: Software, Firmware, and Information Integrity*. Control catalog context; this project does not independently satisfy a control. Official NIST publication.

PCI Security Standards Council, *PCI DSS document library*. Consult current official materials and assess applicability with the responsible compliance owner; this project does not establish PCI DSS compliance. Official PCI SSC library.

Portfolio usage

This PDF describes source code and test evidence in the accompanying repository. Before publication, add the six current application screenshots listed in Section 6, captured from the working app with a controlled test directory and no personal file paths or secrets.